

TP Guidé: Créer et Manipuler des modèles Django

Dans ce TP, nous allons passer d'un site statique à une application web dynamique en introduisant : les modèles Django, la base de données SQLite, les opérations CRUD, l'authentification et la gestion des utilisateurs.

À l'issue de ce TP, vous serez capables de :

- Créer et manipuler des modèles Django ;
- Effectuer des migrations ;
- Interagir avec une base de données SQLite ;
- Implémenter les fonctionnalités CRUD ;
- Gérer les utilisateurs et l'authentification ;
- Afficher dynamiquement les données dans les templates ;
- Structurer une application Django.

Sujet du TP

Développement d'une mini application web : "Gestion des formations"

Chaque formation possède :

- Un titre ;
- Une description ;
- Une durée ;
- Une date de création.

L'application doit permettre :

- D'ajouter une formation ;
- D'afficher la liste des formations ;
- De modifier une formation ;
- De supprimer une formation ;
- De créer un compte utilisateur ;
- De se connecter ;
- De se déconnecter.

I. Créer une application nommée "formations"

II. Création du modèle

1. Créer la classe Formation

Dans ***formations/models.py*** Créer le modèle suivant

```
from django.db import models

class Formation(models.Model):
    titre = models.CharField(max_length=100)

    description = models.TextField()
```

```
duree = models.IntegerField()

date_creation = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return self.titre
```

III. Migrations

1. Générer les migrations

```
python manage.py makemigrations
```

2. Appliquer les migrations :

```
python manage.py migrate
```

IV. Administration Django

1. Enregistrer le modèle dans **admin.py**

```
# Importe le module d'administration natif de Django
from django.contrib import admin

# Importe le modèle 'Formation' depuis le fichier 'models.py' du dossier courant (.)
from .models import Formation

# Enregistre le modèle 'Formation' pour le rendre visible, consultable
# et modifiable depuis l'interface d'administration Django (/admin)
admin.site.register(Formation)
```

2. Créer un super utilisateur via anaconda prompt

```
python manage.py createsuperuser
```

3. Tester l'administration

Ouvrir :

```
http://127.0.0.1:8000/admin
```

Ajouter quelques informations.

V. Affichage des données (READ)

1. Créer la vue :

Dans formations/views.py :

```
from django.shortcuts import render

# Importe le modèle 'Formation' pour pouvoir interroger la base de données
from .models import Formation
```

```
# Définition de la vue sous forme de fonction. Elle prend 'request' (la requête HTTP)
en paramètre obligatoire.
def liste_ formations(request):

    # Effectue une requête SQL pour récupérer TOUTES les lignes de la table Formation
    en base de données
    formations = Formation.objects.all()

    # Renvoie une réponse HTTP en générant la page HTML finale
    return render(
        request,          # La requête HTTP d'origine
        'formations/liste.html', # Le chemin vers le fichier HTML servant de template
        {
            'formations': formations # Le dictionnaire (contexte) pour envoyer les données
            au template HTML
        }
    )
```

VI. Création du template

Dans **templates/formations/liste.html** :

```
<!-- Indique que cette page utilise le squelette général du site -->
{% extends 'base.html' %}
<!-- Ouvre la zone d'injection du contenu principal -->
{% block content %}

<h1>Liste des formations</h1>

<!-- Génère dynamiquement l'URL à partir du nom ('name') défini dans urls.py -->
<a href="{% url 'ajouter_formation' %}">
    Ajouter une formation
</a>

<br><br>
<!-- Boucle qui parcourt chaque formation envoyée par la vue Django -->
{% for formation in formations %}

    <div>

        <h2>{{ formation.titre }}</h2>

        <p>{{ formation.description }}</p>

        <p>

            <strong>Durée :</strong>

            {{ formation.duree }} heures

        </p>
```

```
<!-- Génère l'URL de modification en lui transmettant l'ID de la formation
actuelle comme paramètre -->
<a href="{% url 'modifier_formation' formation.id %}">
    Modifier
</a>

<!-- Génère l'URL de suppression en lui transmettant l'ID de la formation actuelle
comme paramètre -->
<a href="{% url 'supprimer_formation' formation.id %}">
    Supprimer
</a>

</div>

{% endfor %}

{% endblock %}
```

VII. CREATE (Ajout d'une formation)

1. Créer un formulaire

Ajouter le fichier **forms.py** dans l'application formations :

```
# Importe le module de gestion des formulaires de Django
from django import forms

# Importe le modèle 'Formation' pour pouvoir lier le formulaire à la table de la base de
données
from .models import Formation

# Déclarations d'un formulaire de type ModelForm (généré automatiquement à partir
d'un modèle)
class FormationForm(forms.ModelForm):

    # Sous-classe de configuration obligatoire pour indiquer à Django comment
construire le formulaire
    class Meta:

        # Spécifie le modèle de référence qui sera utilisé pour créer les champs
        model = Formation

        # Indique à Django d'inclure TOUS les champs du modèle (titre, description,
duree, etc.) dans le formulaire
        fields = "__all__"
```

2. Créer la vue d'ajout

```
# Importe 'render' pour afficher le HTML et 'redirect' pour rediriger l'utilisateur vers
une autre page
from django.shortcuts import render, redirect

# Importe le formulaire 'FormationForm' créé précédemment
from .forms import FormationForm

# Vue qui gère à la fois l'affichage du formulaire vide et la soumission des données
def ajouter_formation(request):
    # Raccourci Django crucial :
    # Si la requête est en POST (formulaire soumis), il remplit le formulaire avec les
    données reçues (request.POST).
    # Sinon (premier affichage de la page), il crée un formulaire vide (None).
    form = FormationForm(request.POST or None)

    # Vérifie si le formulaire a été soumis et si toutes les données respectent les règles
    de validation (champs obligatoires, types, etc.)
    if form.is_valid():

        # Enregistre automatiquement la nouvelle formation directement dans la base de
        données
        form.save()

        # Redirige immédiatement l'utilisateur vers la page de la liste des formations pour
        éviter les doubles soumissions
        return redirect('liste_formations')

    # Si le formulaire est vide ou s'il contient des erreurs, Django affiche (ou réaffiche)
    la page HTML
    return render(
        request,
        'formations/ajouter.html', # Le template qui contient le formulaire HTML
        {
            'form': form # Envoie l'instance du formulaire (avec ou sans erreurs) au
            template
        }
    )
```

3. Créer le template

```
{% extends 'base.html' %}

{% block content %}

<h1>Ajouter une formation</h1>

<!-- Déclare un formulaire qui envoie les données de manière sécurisée via la méthode
HTTP POST -->
<form method="POST">
```

```
<!-- Jeton de sécurité obligatoire pour protéger le formulaire contre les attaques
CSRF (failles de sécurité) -->
{% csrf_token %}

<!-- Génère et affiche automatiquement tous les champs du formulaire enveloppés
dans des balises de paragraphe (<p>) -->
{{ form.as_p }}

<!-- Bouton de type 'submit' qui déclenche l'envoi des données du formulaire vers la
vue Django -->
<button type="submit">

    Ajouter

</button>

</form>

{% endblock %}
```

VIII. UPDATE (Modification)

1. Créer la vue

```
# Importe le raccourci permettant de récupérer un objet ou de renvoyer une erreur 404 si
l'identifiant n'existe pas
from django.shortcuts import get_object_or_404

# Vue prenant en paramètres la requête HTTP et l'ID de la formation à modifier (transmis via
l'URL)
def modifier_formation(request, id):

    # Cherche la formation avec l'ID demandé. Si elle n'existe pas en base de données, affiche
une page "404 Not Found"
    formation = get_object_or_404(
        Formation,
        id=id
    )

    # Associe le formulaire à l'instance récupérée :
    # - Si la requête est en POST, Django met à jour les données existantes de cette instance.
    # - Sinon, il pré-remplit les champs du formulaire avec les données actuelles de la
formation.
    form = FormationForm(
        request.POST or None,
        instance=formation
    )

    # Vérifie si les modifications apportées respectent les règles de validation du formulaire
    if form.is_valid():
```

```
# Enregistre les modifications directement sur la ligne existante en base de données (ne crée pas de doublon)
form.save()

# Redirige l'utilisateur vers la liste globale des formations après la sauvegarde réussie
return redirect('liste_ formations')

# Affiche le template de modification en lui transmettant le formulaire pré-rempli (ou contenant les erreurs de saisie)
return render(
    request,
    'formations/modifier.html',
    {
        'form': form
    }
)
```

IX. DELETE (Suppression)

1. Créer la vue

```
# Vue qui gère la suppression d'une formation, prenant en paramètres la requête HTTP et l'ID de la cible
def supprimer_formation(request, id):

    # Tente de récupérer la formation correspondante à l'ID en base de données ou renvoie une erreur 404 si elle est introuvable
    formation = get_object_or_404(
        Formation,
        id=id
    )

    # Exécute la requête SQL 'DELETE' pour supprimer définitivement cet enregistrement de la base de données
    formation.delete()

    # Redirige immédiatement l'utilisateur vers la page de la liste des formations mise à jour
    return redirect('liste_ formations')
```

X. Gestion des utilisateurs

1. Créer une application utilisateurs
2. Créer le formulaire d'inscription

```
# Importe le formulaire d'inscription natif de Django (qui gère déjà le hachage et la validation des mots de passe)

from django.contrib.auth.forms import UserCreationForm

# Importe le modèle d'utilisateur par défaut de Django pour la base de données
```

```
from django.contrib.auth.models import User

# Déclarations d'un formulaire personnalisé basé sur le système de création d'utilisateur de Django

class RegisterForm(UserCreationForm):

    # Sous-classe de configuration pour lier le formulaire au modèle User et choisir les champs

    class Meta:

        # Spécifie que ce formulaire va créer un enregistrement dans la table du modèle User

        model = User

        # Liste les champs à afficher à l'écran pour l'inscription :

        # - 'username' : le nom d'utilisateur unique

        # - 'email' : l'adresse de messagerie

        # - 'password1' et 'password2' : les deux champs de saisie du mot de passe pour vérification

        fields = [

            'username',

            'email',

            'password1',

            'password2'

        ]
```

3. Créer la vue d'inscription

```
# Importe 'render' pour afficher la page HTML et 'redirect' pour rediriger l'utilisateur après l'inscription

from django.shortcuts import render, redirect

# Importe le formulaire personnalisé d'inscription 'RegisterForm' que vous avez créé

from .forms import RegisterForm

# Vue qui gère l'affichage du formulaire et le traitement de la création de compte

def register(request):

    # Initialise le formulaire : il extrait les données si la requête est en POST, sinon il crée un formulaire d'inscription vide

    form = RegisterForm(request.POST or None)

    # Déclenche les validations de sécurité (vérification du nom d'utilisateur, de l'email et de la conformité des mots de passe)
```



```
if form.is_valid():

    # Enregistre le nouvel utilisateur en base de données en hachant de manière sécurisée son mot
    de passe

    form.save()

    # Redirige l'utilisateur vers la page de connexion (nommée 'login' dans votre fichier urls.py)

    return redirect('login')


# Génère la page HTML finale en y injectant le formulaire (vide ou contenant les messages
d'erreurs de validation)

return render(

    request,

    'utilisateurs/register.html', # Chemin vers le fichier template de votre application

    {

        'form': form # Rnd le formulaire accessible dans le fichier HTML via la variable {{ form }}

    }

)
```

Travail à faire :

1. Créer les applications nécessaires.
2. Créer le modèle Formation.
3. Réaliser les migrations.
4. Ajouter des données via l'administration.
5. Implémenter toutes les opérations CRUD.
6. Créer les templates HTML.
7. Ajouter un fichier CSS.
8. Implémenter :
 - Inscription ;
 - Connexion ;
 - Déconnexion.
9. Tester toutes les fonctionnalités.
10. Organiser correctement le projet Django.